

01 TELEMETRY RECEIVER Specification

- [SMC Telemetry Receiver — Functional Specification](#)
 - [Table of contents](#)
 - [1. Overview](#)
 - [2. Address map and integration](#)
 - [2.1 Base address \(RTL-aligned\)](#)
 - [3. Register map](#)
 - [3.1 CTRL_\(0x00\) bit fields](#)
 - [3.2 STATUS_\(0x04\)](#)
 - [3.3 Interrupt registers \(0x08 / 0x0C / 0x10\)](#)
 - [4. ATB message format](#)
 - [4.1 Worked example](#)
 - [5. Behaviour and flow diagrams](#)
 - [5.1 Datapath](#)
 - [5.2 Beat arrival](#)
 - [5.3 Queue overflow \(drop-oldest\)](#)
 - [5.4 Software drain loop](#)
 - [6. Interrupt model](#)
 - [6.1 BUFFER_THRESHOLD — level](#)
 - [6.2 MISSING_LAST — sticky](#)
 - [6.3 INTR_TEST_\(0x10\)](#)
 - [6.4 debug_o](#)
 - [7. Flush operations](#)
 - [7.1 RX flush \(CTRL.TELEMTRY_RX_FLUSH\)](#)
 - [7.2 TX flush \(CTRL.TELEMTRY_TX_FLUSH, ATB AF channel\)](#)
 - [8. Reset](#)
 - [9. Modeling scope and abstractions](#)

SMC Telemetry Receiver — Functional Specification

Externally-observable behaviour of the SystemC/TLM-2.0 LT model in `smc/peripherals/telemetry_receiver`. The authoritative hardware sources are `hw/comp/telemetry_receiver/data/registers/rdl/telemetry_receiver.rdl` (register map) and `hw/comp/telemetry_receiver/rtl/telemetry_receiver.sv` (behaviour), with ATB widths from `hw/comp/telemetry_receiver/rtl/telemetry_receiver_pkg.sv`.

Table of contents

1. [Overview](#)
 2. [Address map and integration](#)
 3. [Register map](#)
 4. [ATB message format](#)
 5. [Behaviour and flow diagrams](#)
 6. [Interrupt model](#)
 7. [Flush operations](#)
 8. [Reset](#)
 9. [Modeling scope and abstractions](#)
-

1. Overview

The Telemetry Receiver is the **sink of the SoC telemetry path**. A telemetry transmitter elsewhere in the SoC samples hardware counters and streams them to the receiver over an **ATB** (AMBA Trace Bus) byte interface. The receiver:

1. re-assembles the byte stream into 64-bit **packets** and packets into **messages** (a probe ID plus up to 32 counter samples);
2. queues completed messages in a small **circular message buffer**;
3. exposes the **oldest** queued message to software through a read-only register aperture, which software drains one message at a time;
4. raises an interrupt when the queue passes a software-programmed **fill threshold**, or when a malformed message is received (**missing-last**).

It is passive on the register bus: it never initiates register traffic. Its only outbound activity is a **flush request** to the transmitter over the ATB AF channel.

The model reproduces:

- the **32-bit register file** (bit-compatible with the RDL), including the read-only “oldest message” view and the write-pulse `CTRL` bits;
 - **ATB assembly and message decode**, including per-counter valid bits and messages that span multiple packets;
 - the **circular message buffer** with the RTL’s **drop-oldest** overflow policy;
 - both **interrupt sources** with their different flavours (level vs. sticky) and the `INTR_TEST` forcing paths;
 - **RX flush** (discard everything buffered) and the **TX flush handshake** toward the transmitter.
-

2. Address map and integration

Each receiver instance occupies a `0x100` register window (`telemetry_receiver_wrap.rdl` stride). The SMC wrapper instantiates `NUM_TELEMETRY_RECEIVERS = 3` instances back-to-back, so the wrapper spans `0x300`:

Instance	Window (relative to the wrapper base)
0	+0x000 .. +0x0FF
1	+0x100 .. +0x1FF
2	+0x200 .. +0x2FF

Access is AXI4-Lite-style, **32-bit** (`regwidth = accesswidth = 32`).

2.1 Base address (RTL-aligned)

Each receiver instance occupies a `0x100` register window (`telemetry_receiver_wrap.rdl` stride). The SMC wrapper instantiates `NUM_TELEMETRY_RECEIVERS = 3` instances back-to-back, so the wrapper spans `0x300`:

Instance	Absolute base (<code>smc_top.rdl</code>)
0	<code>0xC000_9000</code>
1	<code>0xC000_9100</code>
2	<code>0xC000_9200</code>

`smc-vp` routes the wrap at `0xC000_9000` (Option A — match silicon). I2C was relocated to its RTL base `0xC000_5000`, and I3C to `0xC003_A000`, to free `0xC000_9000`. The model itself still decodes **window-relative offsets only**; the platform address router supplies the absolute base.

3. Register map

All registers are 32-bit and accessed with 4-byte, 4-byte-aligned transactions.

Offset	Name	SW access	Reset	Description
0x00	CTRL	rw	0x0	Buffer pop, RX/TX flush, fill threshold (see §3.1).
0x04	STATUS	ro	0x1	[0] BUFFER_EMPTY, [4] BUFFER_FULL.
0x08	INTR_STATUS	rw	0x0	[0] MISSING_LAST (sticky, W1C), [4] BUFFER_THRESHOLD (read-only level).
0x0C	INTR_ENABLE	rw	0x0	[0] MISSING_LAST, [4] BUFFER_THRESHOLD.
0x10	INTR_TEST	rw	0x0	[0] MISSING_LAST (write pulse), [4] BUFFER_THRESHOLD (stored level).
0x14	TELEMETRY_PROBE_ID	ro	0x0	[4:0] probe ID of the oldest queued message.
0x18	TELEMETRY_COUNTER_VLDS	ro	0x0	[31:0] per-counter valid bits of the oldest queued message.
0x80 + 4· <i>i</i>	TELEMETRY_COUNTER[<i>i</i>], <i>i</i> = 0..31	ro	0x0	Counter <i>i</i> of the oldest queued message.

Offsets inside the 0x100 window that decode to no register — the gap [0x1C, 0x80) — return TLM_ADDRESS_ERROR_RESPONSE, as do offsets $\geq 0x100$ and misaligned accesses.

TELEMETRY_COUNTER[*i*] for $i \geq \text{max_counters_per_message}$ reads 0 with its valid bit clear: the RTL ties off the unused counter registers, and all 32 are always readable regardless of the configured message size.

3.1 CTRL (0x00) bit fields

bit: 31..24	23.....12	11..9	8	7..5	4	3..1	0
+	+	+	+	+	+	+	+
rsvd	BUFFER_THRESHOLD	rsvd	TX_FLUSH	rsvd	RX_FLUSH	rsvd	BUFFER_POP
+	+	+	+	+	+	+	+

Field	Bits	Access	Behaviour
<code>BUFFER_POP</code>	[0]	wo, self-clearing pulse	Dequeues the oldest message. Ignored when the queue is empty. Always reads 0.
<code>TELEMETRY_RX_FLUSH</code>	[4]	wo, self-clearing pulse	Discards the message queue and any partially assembled message. Always reads 0.
<code>TELEMETRY_TX_FLUSH</code>	[8]	rw, hardware-cleared	Requests a transmitter flush; reads back as 1 while the request is outstanding, and is cleared by hardware on <code>afready_i</code> (§7.2).
<code>BUFFER_THRESHOLD</code>	[23:12]	rw	Fill level above which the <code>BUFFER_THRESHOLD</code> interrupt asserts (§6.1).

Firmware note. `CTRL` is a single register, so a read-modify-write is required to pulse `BUFFER_POP` or `RX_FLUSH` without clobbering `BUFFER_THRESHOLD` (or an outstanding `TX_FLUSH`). Writing a bare `CTRL = BUFFER_POP` also zeroes the threshold, which will typically leave the threshold interrupt asserted (`fill > 0`).

3.2 STATUS (0x04)

Field	Bits	Meaning
<code>BUFFER_EMPTY</code>	[0]	Message queue holds no messages. Set at reset.
<code>BUFFER_FULL</code>	[4]	Message queue holds <code>buffer_depth</code> messages; the next completed message will drop the oldest one.

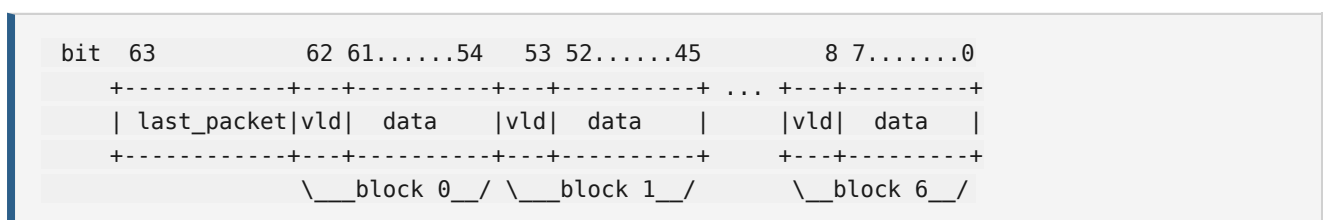
Both are hardware-driven; writes are accepted and ignored.

3.3 Interrupt registers (0x08 / 0x0C / 0x10)

All three share one bit layout — bit [0] `MISSING_LAST`, bit [4] `BUFFER_THRESHOLD` — so an enable mask can be applied to any of them. Reserved bits are never stored and read 0.

4. ATB message format

The transmitter sends **byte beats**. Eight beats form one 64-bit **packet**; one or more packets form a **message**. A packet is a bit-packed `{last_packet, blocks[0..6]}`:



- A **block** is `{vld:1, data:8}` — one payload byte plus its valid bit. Seven blocks fill 63 bits, leaving bit 63 for `last_packet`.
- **Beat ordering:** beat i of a packet carries packet bits `[8i+7 : 8i]`. The first beat on the wire is therefore the packet **LSB**, and the eighth (last) beat carries the `last_packet` marker.
- `blocks[0]` of the **first** packet is the message **header**: the probe ID sits at packet bits `[60:56]` (i.e. `blocks[0].data[7:2]`).
- Every subsequent block is counter payload, **most-significant byte first**, four blocks per 32-bit counter, running **across packet boundaries**.
- A counter is **valid only if all four of its blocks are valid**. An invalid counter reads back as `0` with its `TELEMETRY_COUNTER_VLDS` bit clear.

A message therefore occupies

```
packets_per_message = ceil((1 + 4 * max_counters_per_message) / 7)
beats_per_message   = 8 * packets_per_message
```

<code>max_counters_per_message</code>	Blocks	Packets	Beats
1	5	1	8
4	17	3	24
8	33	5	40
32	129	19	152

4.1 Worked example

`probe_id = 0x15`, one counter per message, `counter[0] = 0xDEAD_BEEF` (valid):

Block	Content	Packet bits
0	header, <code>data = probe_id << 2 = 0x54</code>	<code>[62:54]</code>
1	<code>0xDE</code>	<code>[53:45]</code>
2	<code>0xAD</code>	<code>[44:36]</code>
3	<code>0xBE</code>	<code>[35:27]</code>
4	<code>0xEF</code>	<code>[26:18]</code>
5, 6	unused (invalid)	<code>[17:0]</code>

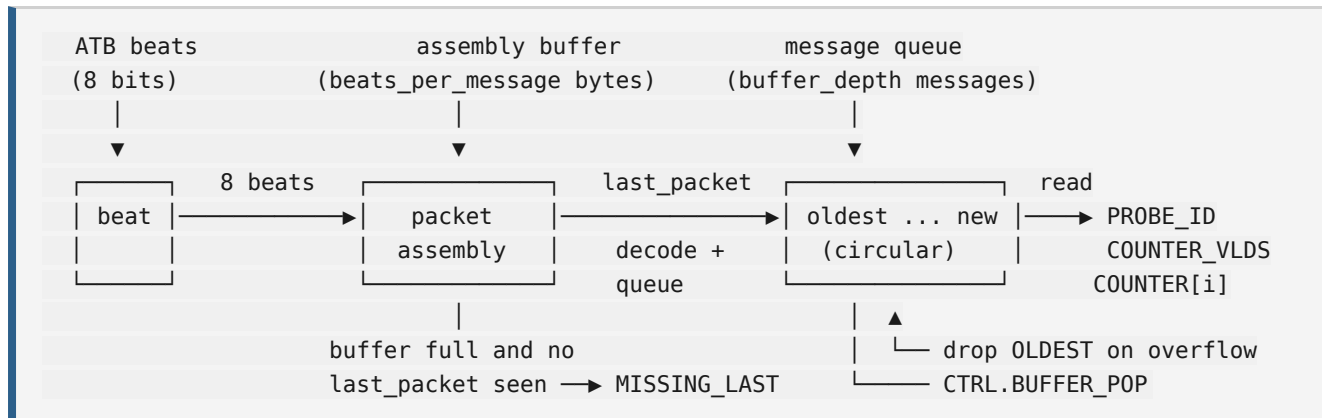
Packet word = `0xD53B_DADD_F7BC_0000`, sent as the beats

```
0x00, 0x00, 0xBC, 0xF7, 0xDD, 0xDA, 0x3B, 0xD5      (first .. last)
```

This exact stream is hard-coded in `test/telemetry_receiver_tb.cpp` as a decode oracle, so the decoder is not validated by its own encoder.

5. Behaviour and flow diagrams

5.1 Datapath



5.2 Beat arrival

```

push beat:
  if in reset          -> reject (atready_o low)
  append beat to assembly buffer
  if (beats % 8) != 0   -> done          # packet incomplete
  packet = assemble 8 beats
  if last_packet(packet):
    message = decode(assembly buffer)    # probe id + counters
    queue(message)                       # drops oldest if full
    restart assembly buffer
  else if assembly buffer full:
    MISSING_LAST event                   # malformed: no marker in time
    discard partial message
    restart assembly buffer
  
```

The receiver **recovers automatically** from a missing-last event: the assembly buffer restarts, so the next well-formed message decodes normally.

5.3 Queue overflow (drop-oldest)

When a message completes while the queue already holds `buffer_depth` messages, the RTL advances the read pointer — the **oldest** message is discarded and the new one is stored. Telemetry is therefore always the *freshest* available, and software that falls behind loses history rather than current data.

5.4 Software drain loop

```
while (!(readl(TELEMETRY_BASE + STATUS) & STATUS_BUFFER_EMPTY)) {
    uint32_t probe = readl(TELEMETRY_BASE + TELEMETRY_PROBE_ID);
    uint32_t vlids = readl(TELEMETRY_BASE + TELEMETRY_COUNTER_VLDS);
    for (int i = 0; i < NUM_COUNTERS; i++)
        if (vlids & (1u << i))
            consume(probe, i, readl(TELEMETRY_BASE + TELEMETRY_COUNTER(i)));

    /* Read-modify-write: preserve BUFFER_THRESHOLD while pulsing the pop. */
    writel(TELEMETRY_BASE + CTRL,
           (readl(TELEMETRY_BASE + CTRL) & CTRL_THRESHOLD_MASK) | CTRL_BUFFER_POP);
}
```

While the queue is empty, `TELEMETRY_PROBE_ID`, `TELEMETRY_COUNTER_VLDS` and all `TELEMETRY_COUNTER[i]` read 0.

6. Interrupt model

`irq_o` is the **OR of the two enabled sources**. Both are reported in `INTR_STATUS`, but they behave differently.

6.1 BUFFER_THRESHOLD — level

```
threshold_cmp = CTRL.BUFFER_THRESHOLD & ((1 << (clog2(buffer_depth) + 1)) - 1)
raw           = (fill_level > threshold_cmp) || INTR_TEST.BUFFER_THRESHOLD
irq           = raw && INTR_ENABLE.BUFFER_THRESHOLD
INTR_STATUS.BUFFER_THRESHOLD = irq           (read-only mirror)
```

- Strictly **greater than**: a threshold of N fires at fill $N+1$. Threshold 0 therefore fires on any queued message.
- The status bit is a **read-only mirror**, not a latch: writing 1 to it does not clear it. It de-asserts only when the fill drops back to or below the threshold, the enable is cleared, or `INTR_TEST` is released.
- `CTRL.BUFFER_THRESHOLD` is a 12-bit field, but the RTL casts it to the message-buffer pointer width ($\text{clog2}(\text{buffer_depth}) + 1$ bits) before comparing, so larger values **truncate**. With `buffer_depth = 8` the compare width is 4 bits and a programmed threshold of `0x15` behaves as `0x5`.

6.2 MISSING_LAST — sticky

```
event = (assembly buffer filled without last_packet) || write INTR_TEST.MISSING_LAST
if (event && INTR_ENABLE.MISSING_LAST) INTR_STATUS.MISSING_LAST = 1 # sticky
irq = INTR_STATUS.MISSING_LAST
```

- The enable is sampled **at the moment of the event**: if the enable is clear when the malformed message is detected, no status is latched (the event is lost, not deferred). Enabling the interrupt afterwards does not resurrect it.
- Cleared by **writing 1** to `INTR_STATUS[0]` (W1C).
- The `debug_o[0]` indication is set regardless of the enable, so the event is still observable in a waveform / debug dump when the interrupt is masked.

6.3 INTR_TEST (0x10)

Bit	Flavour	Effect
[0] MISSING_LAST	write pulse (never reads back)	Injects one missing-last event, subject to the same enable gating as a real one.
[4] BUFFER_THRESHOLD	stored level	Forces the threshold source while set; writing 0 releases it.

6.4 debug_o

A 4-bit observability vector (`telemetry_receiver.sv debug_o`):

Bit	Name	Meaning
[0]	MISSING_LAST	A missing-last event occurred (latched until RX flush / reset).
[1]	BUFFER_FULL	Message queue full.
[2]	BUFFER_EMPTY	Message queue empty.
[3]	ASSEMBLY_FULL	The assembly buffer filled without a <code>last_packet</code> (latched alongside bit 0).

7. Flush operations

7.1 RX flush (CTRL.TELEMETRY_RX_FLUSH)

Discards **both** the message queue and any partially assembled message, and clears the latched `debug_o` event bits. `STATUS.BUFFER_EMPTY` is set afterwards. The next beat starts a fresh message, so a flush is the way to resynchronise after a stream error.

If a single `CTRL` write sets both `RX_FLUSH` and `BUFFER_POP`, the **flush wins** (the whole queue is discarded, not just the oldest entry).

7.2 TX flush (CTRL.TELEMETRY_TX_FLUSH, ATB AF channel)

```
SW writes CTRL.TX_FLUSH=1 → avalid_o = 1, CTRL.TX_FLUSH reads 1
                             (request outstanding)
transmitter asserts aready_i → avalid_o = 0, CTRL.TX_FLUSH self-clears
```

This asks the transmitter to push out whatever it holds, so software can drain in-flight telemetry. If `aready_i` is already high when the bit is written, the request retires immediately and `CTRL.TX_FLUSH` reads back 0.

8. Reset

`rst_n_i` is an **active-low asynchronous** reset. While it is low:

- `aready_o` de-asserts and **ATB beats are refused**;

- all registers return to their reset values (`STATUS = BUFFER_EMPTY` , all others 0);
- the assembly buffer, the message queue, the latched debug bits, and any outstanding TX-flush request are cleared;
- `irq_o` and `afvalid_o` de-assert.

Nothing survives reset: there is no retained or sticky state across it.

9. Modeling scope and abstractions

Modeled faithfully

- Register map, offsets, access types, masks, and reset values (per the RDL).
- ATB packet/block bit packing, beat ordering, probe-ID placement, MSB-first counter bytes, and per-counter validity, including messages spanning packets.
- Assembly-buffer sizing, missing-last detection, and automatic recovery.
- Circular queue with drop-oldest overflow; the oldest-message software view.
- Both interrupt sources with their level / sticky flavours, enable gating and `INTR_TEST` forcing, including threshold truncation.
- RX flush, TX flush handshake, and reset behaviour.

Abstracted away

- **No ATB initiator.** There is no transmitter model; beats are injected through the test-bench back door `push_atb_beat()` / `push_atb_beats()` , the same abstraction `beu` uses for error sources and `i2c_controller` for its bus. Wiring a real ATB port is a straightforward extension, and `telemetry_encode_message()` already provides the transmitter-side packing.
- **No cycle-level timing.** `b_transport` is loosely timed and never calls `wait()` ; a single `access_delay_ns` scalar approximates the AXI4-Lite register latency. ATB throughput and back-pressure are not modeled — `atready_o` reflects reset only.
- **Single-cycle RTL artefacts.** `atready_o`'s one-cycle dip during a flush, and the single-cycle `debug_o` pulses, have no cycle to occupy in an LT model: the flush completes inside the register write, and the pulse bits are latched (until RX flush or reset) so they remain checkable.
- **Registered pipeline stages** of the RTL (`end_of_packet_q` and friends) are evaluated per injected beat. The observable result — which beat completes a message — is identical.

Not present in the SMC configuration

- The RTL's clock-gating and DFT hooks.
- The wrapper's instance-select decode (the platform's address router provides the per-instance base; see §2.1).